

PBA: 8. Prediction and Iteration

Fall 2025

Jaewon (“Jay-one”) Yoo

National Tsing Hua University

Roadmap

1. Prediction
2. Loops
3. Evaluating the Predictions
4. Time-Series Plot
5. Review: Causal Effect Estimation

1/ Prediction

2016 US Presidential Election

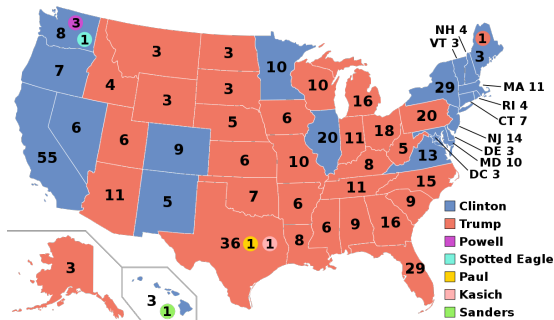
- Why? $\rightsquigarrow$ Huge business impact with managerial/socio-economic implications (e.g., support for minimum wage policy)



Source: [Where They Stand on Economic Issues](#) by WSJ

- 2016 election popular vote:
 - H. Clinton: 65,853,516 (48.2%)
 - D. Trump: 62,984,825 (46.1%)
- Why did Trump win? **Electoral college!**
 - Trump = 304 pt; Clinton = 227 pt
- Election determined by 77,744 votes (margins in WI, MI, and PA)
 - 0.056% of all eligible voters (~136 million)

Predicting Presidential Election Outcomes



- **Electoral college system:**

- Must win an absolute majority of 538 electoral votes.
- $538 = 435 \text{ (House of Representatives)} + 100 \text{ (Senators)} + 3 \text{ (DC)}$
- Must win at least 270 votes
- Nobody wins an absolute majority $\rightsquigarrow$ House vote

- Implication: **must predict winner of each state!**

Prediction Strategy

- Predict state-level support for each candidate using polls/surveys
- Allocate electoral college votes of that state to its predicted winner
- Aggregate EC votes across states to determine the predicted winner
- Coding strategy:
 1. For each state, subset to polls within that state.
 2. Further subset the latest polls (i.e., latest information available).
 3. Average the latest polls to estimate support for each candidate.
 4. Allocate the electoral votes to the candidate who has greatest support.
 5. Repeat this for all states and aggregate the EC votes.

↪ Sounds like a lot of subsets!... What to do?

2/ Loops

A Simple Example

Cooperative congressional election study (i.e., CCES; see [Wikipedia](https://cces.gov.harvard.edu/) or <https://cces.gov.harvard.edu/>)

- Q: What if we wanted to know the number of unique values of each column of the `cces_2020` data?

```
1 > # f=factor; d=double/numeric
2 > cces_2020 <- read_csv("https://bit.ly/49AZaAM", col_types = "ffffdf"); cces_2020
3
4 # A tibble: 51,551 × 6
5   gender race  educ          pid3      turnout_self pres_vote
6   <fct> <fct> <fct>          <fct>          <dbl> <fct>
7   1 Male   White 2-year      Republican      1 Donald J. Trump (Republican)
8   2 Female White Post-grad Democrat        NA NA
9   3 Female White 4-year      Independent      1 Joe Biden (Democrat)
10  4 Female White 4-year      Democrat        1 Joe Biden (Democrat)
11  5 Male   White 4-year      Independent      1 Other
12  6 Male   White Some college Republican      1 Donald J. Trump (Republican)
13  7 Male   Black Some college Not sure        NA NA
14  8 Female White Some college Independent      1 Donald J. Trump (Republican)
15  9 Female White High school graduate Republican      1 Donald J. Trump (Republican)
16 10 Female White 4-year      Democrat        1 Joe Biden (Democrat)
17 # 51,541 more rows
18 # Use `print(n = ...)` to see more rows
```

Manually Changing Values

- Manually 'iterate' through each column:

```
1 > length(unique(cces_2020$gender))
2 [1] 2
3 > length(unique(cces_2020$race))
4 [1] 8
5 > length(unique(cces_2020$educ))
6 [1] 6
7
8 # Repeat for the rest of the columns, i.e., pid3, turnout_self, pres_vote.
```

- Subsetting with brackets: recall we can also access variables with [[]].

```
1 > unique(cces_2020$gender) # Point to the first column, gender
2 [1] Male   Female
3 Levels: Male Female
4 > unique(cces_2020[[1]])
5 [1] Male   Female
6 Levels: Male Female
7
8 > unique(cces_2020$pid3) # Point to the 4th column, pid3
9 [1] Republican Democrat   Independent Not sure   Other
10 Levels: Republican Democrat Independent Not sure Other
11 > unique(cces_2020[[4]])
12 [1] Republican Democrat   Independent Not sure   Other
13 Levels: Republican Democrat Independent Not sure Other
```

Manually Changing Values, Alternative

```
1 > length(unique(cces_2020[[1]])) # No. unique values for 1st col.  
2 [1] 2
```

```
1 > length(unique(cces_2020[[2]])) # No. unique values for 2nd col.  
2 [1] 8
```

⋮

```
1 > length(unique(cces_2020[[5]])) # No. unique values for (k-1)th col.  
2 [1] 3
```

```
1 > length(unique(cces_2020[[6]])) # No. unique values for kth col.  
2 [1] 7
```

Recognizing the 'Template'

What if you had more values? $\rightsquigarrow$ Not efficient!

Recognize the template/recurring pattern:

```
1 length(unique(cces_2020[[<<column number>>]]))
```

Can we give R this template and a set of column numbers have it do our task repeatedly?

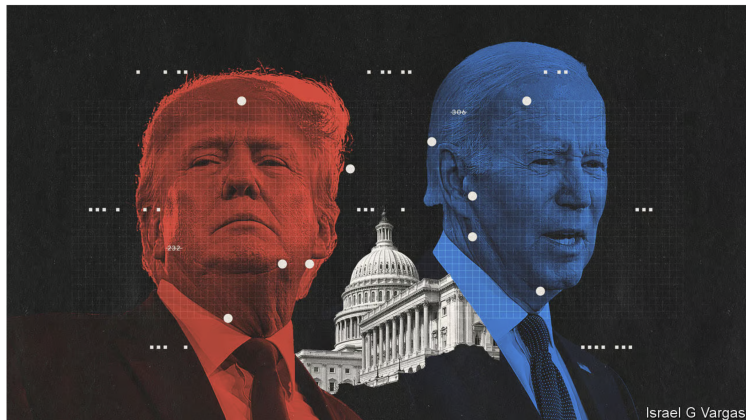
Loops in R

for loop provide a way to execute these templates multiple times:

```
1 > output <- rep(NA, times = ncol(cces_2020)) # 1. output
2 > for (i in seq_along(cces_2020)) { # 2. sequence
3   output[i] <- length(unique(cces_2020[[i]])) # 3. body
4 }
5
6 > output
7 [1] 2 8 6 5 3 7
```

- Elements of a loop:
 1. **output**: vector to hold the output.
 2. **i**: placeholder name we'll use to swap values between iterations.
 3. **seq_along(cces_2020)**: vec. of values for the placeholder to take.
 4. **body**: a set of expressions that will be repeatedly evaluated.
 5. **{ }**: curly braces to define beginning and end of the loop.
- Indentation is important for readability of the code.
- Recall: predict winner of each state to predict election outcome.
 - What to do? $\rightsquigarrow$ **State-level polling data**

2020 US Presidential Election



Source: Donald Trump and Joe Biden both want to run again by Economists

2020 Polling Prediction

- Election data: `pres20`

Name	Description
<code>state</code>	abbreviated name of state
<code>biden</code>	Biden's vote share (percentage)
<code>trump</code>	Trump's vote share (percentage)
<code>ev</code>	number of electoral college votes for the state

- Polling data: `polls20`

Name	Description
<code>state</code>	state in which poll was conducted
<code>end_date</code>	end date the period when poll was conducted
<code>daysleft</code>	number of days between end date and election day
<code>pollster</code>	name of organization conducting poll
<code>sample_size</code>	name of organization conducting poll
<code>biden</code>	predicted support for Biden (percentage)
<code>trump</code>	predicted support for Trump (percentage)

Import and Examine Data

- We will use `polls20` to predict the outcome.
 - Calculate Biden's margin of victory:

```
1 > polls20 <- as_tibble(read_csv(url("https://bit.ly/3R1qKB8")))
2 > polls20 <- polls20 |>
3   mutate(margin = biden - trump)
```

- Use `glimpse()` to take a look at the data:

```
1 > glimpse(polls20)
2 Rows: 2,445
3 Columns: 8
4 $ end_date      <chr> "2020-11-02", "2020-11-02", "2020-11-02", "2020-11-02", "2020-1..
5 $ state         <chr> "FL", "PA", "FL", "FL", "NV", "GA", "SC", "MT", "ME", "AZ", "FL..
6 $ days_left     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ..
7 $ pollster      <chr> "The Political Matrix/The Listener Group", "Susquehanna", "Insi"..
8 $ sample_size   <int> 966, 499, 400, 1054, 1024, 1041, 817, 920, 1024, 610, 1261, 690..
9 $ biden         <dbl> 44.2, 48.4, 47.0, 47.3, 48.4, 45.4, 39.0, 45.0, 52.0, 50.0, 53...
10 $ trump         <dbl> 48.0, 49.2, 48.2, 49.4, 49.1, 49.7, 51.4, 50.0, 40.0, 47.5, 45...
11 $ margin        <dbl> -3.8, -0.8, -1.2, -2.1, -0.7, -4.3, -12.4, -5.0, 12.0, 2.5, 7.5..
```

Some Preprocessing

- `pres20` will be used for evaluation:

```
1 > pres20 <- as_tibble(read.csv(url("https://bit.ly/47b10aU")))
2 > pres20 <- pres20 |>
3   mutate(margin = biden - trump)
4
5 > glimpse(pres20)
6 Rows: 51
7 Columns: 6
8 $ state <chr> "AK", "AL", "AR", "AZ", "CA", "CO", "CT", "DC", "DE", "FL", "GA", "H"..
9 $ ev <int> 3, 9, 6, 11, 55, 9, 7, 3, 3, 29, 16, 4, 6, 4, 20, 11, 6, 8, 8, 11, 1"..
10 $ biden <dbl> 42.77195, 36.56999, 34.77506, 49.36469, 63.48395, 55.01107, 59.26073"..
11 $ trump <dbl> 52.833143, 62.031643, 62.395730, 49.055981, 34.320724, 41.604126, 39"..
12 $ other <dbl> 0.73248408, 0.69918331, 0.25720074, 0.26322139, 0.24392550, 0.161181"..
13 $ margin <dbl> -10.0611910, -25.4616530, -27.6206679, 0.3087096, 29.1632233, 13.406"
```

Reminder of Our Goal

- Coding strategy:
 1. For each state, subset to polls within that state.
 2. Further subset the latest polls (i.e., latest information available).
 3. Average the latest polls to estimate support for each candidate.
 4. Allocate the electoral votes to the candidate who has greatest support.
 5. Repeat this for all states and aggregate the EC votes.

Poll Prediction for Each State

- Get the list of unique state names to iterate over:

```
1 poll_pred <- rep(NA, 51) # generate vector to assign output / ``output holder``  
2 state_names <- sort(unique(polls20$state))
```

- Add labels to the holder and loop:

```
1 # add labels to holder  
2 > names(poll_pred) <- state_names  
3  
4 > for (i in 1:51) {  
5   state_data <- subset(polls20, subset = (state == state_names[i]))  
6   latest <- state_data$days_left == min(state_data$days_left)  
7   poll_pred[i] <- mean(state_data$margin[latest])  
8 }
```

- Examine the output:

```
1 > head(poll_pred) # Margin of victory for Biden in Alaska, Alabama, Arkansas...  
2  
3 AK    AL    AR    AZ    CA    CO  
4 -9.00 -26.00 -23.00 4.25 26.00 11.00
```

Tidyverse Alternative Version

- Calculate the (predicted) marginal support for Biden (vs. Trump) in each state:

```
1 > poll_pred <- polls20 |>
2   group_by(state) |>
3   filter(days_left == min(days_left)) |>
4   summarize(margin_pred = mean(biden - trump)); poll_pred
5
6 # A tibble: 51 × 2
7   state margin_pred
8   <chr>         <dbl>
9 1 AK           -9
10 2 AL          -26
11 3 AR          -23
12 4 AZ           4.25
13 5 CA           26
14 6 CO           11
15 7 CT           22
16 8 DC           89
17 9 DE           22
18 10 FL          0.0800
19 # 41 more rows
20 # Use `print(n = ...)` to see more rows
```

3/ Evaluating the Predictions

Polling/Prediction Errors

- **Prediction error** = actual outcome – predicted outcome

```
1 > poll_pred <- poll_pred |>
2   left_join(pres20) |>
3   mutate(margin = biden - trump) |>
4   mutate(errors = margin - margin_pred); poll_pred
5
6 Joining with `by = join_by(state)`
7 # A tibble: 51 × 8
8   state margin_pred ev biden trump other margin errors
9   <chr>      <dbl> <int> <dbl> <dbl> <dbl>    <dbl> <dbl>
10 1 AK          -9         3  42.8  52.8  0.732 -10.1   -1.06
11 2 AL         -26         9  36.6  62.0  0.699 -25.5    0.538
12 3 AR        -23         6  34.8  62.4  0.257 -27.6   -4.62
13 4 AZ          4.25        11  49.4  49.1  0.263   0.309 -3.94
14 5 CA         26         55  63.5  34.3  0.244  29.2    3.16
15 6 CO         11         9  55.0  41.6  0.161  13.4    2.41
16 7 CT         22         7  59.3  39.2  0.129  20.1   -1.93
17 8 DC         89         3  92.1   5.40  0.491  86.8   -2.25
18 9 DE         22         3  58.7  39.8  0.0780 19.0   -3.03
19 10 FL         0.0800        29  47.9  51.2  0.0835 -3.36   -3.44
20 # 41 more rows
21 # Use `print(n = ...)` to see more rows
```

Assessing the Prediction Error

- **Bias:** average prediction error
 - Direction of the bias (–) $\rightsquigarrow$ survey poll's prediction was 'overestimating' Biden's victory (predicted support > actual support for Biden).

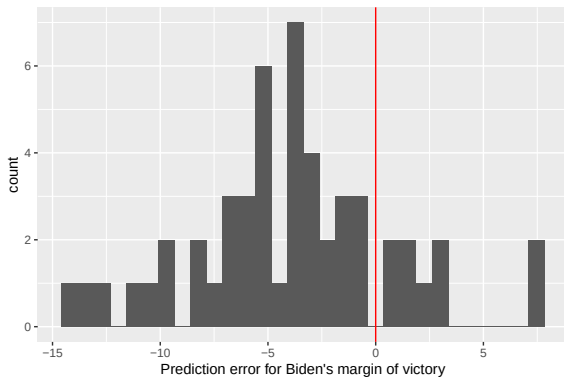
```
1 > mean(poll_pred$errors)  
2 [1] -3.983248
```

- **Root mean-squared error:** average magnitude of the prediction error
 - The poll predictions were off approx. by 6.06%p from the actual results.
 - Determining the magnitude depends on the context:
 - i.e., this is large considering that the election outcome could be flipped by as small as 1%p.

```
1 > sqrt(mean(poll_pred$errors^2))  
2 [1] 6.06975
```

Histogram of the Errors

```
1 > ggplot(poll_pred, aes(x = errors)) +  
2   geom_histogram() +  
3   labs(x = "Prediction error for Biden's margin of victory") +  
4   geom_vline(xintercept = 0, color = "red")
```

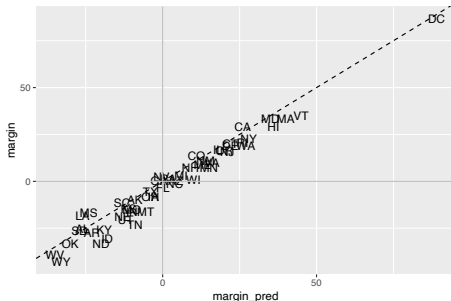


- Prediction error in favor of (i.e., overestimating) Biden's victory.

Comparing Polls to Outcome

- Sometimes we want to plot text labels instead of point and we use `geom_text()` and the `label` aesthetic:
 - Diagonal line representing where predicted margin = actual margin.
 - e.g., Tennessee (TN) below the line: the predicted margin > actual margin of victory for Biden.

```
1 > ggplot(poll_pred, aes(x = margin_pred, y = margin)) +  
2   geom_text(aes(label = state)) +  
3   geom_abline(xintercept = 0, slope = 1, linetype = 2) +  
4   geom_hline(yintercept = 0, color = "grey") +  
5   geom_vline(xintercept = 0, color = "grey")
```



Classification

- Election prediction: need to predict winner in each state.
 - (Actual) number of EC votes Joe Biden won:

```
1 > poll_pred |>  
2   filter(margin > 0) |> ## States in which Biden actually won  
3   summarize(sum(ev)) |> pull() # pull to extract single column  
4 [1] 306
```

- (Predicted) number of EC votes Joe Biden won:

```
1 > poll_pred |>  
2   filter(margin_pred > 0) |> ## States in which Biden predicted to win  
3   summarize(sum(ev)) |> pull()  
4 [1] 328
```

- Prediction of binary outcome variable = **classification problem**
- Wrong prediction $\rightsquigarrow$ misclassification
 1. **True positive:** predict Biden wins when he actually wins.
 2. **False positive:** predict Biden wins when he actually loses.
 3. **True negative:** predict Biden loses when he actually loses.
 4. **False negative:** predict Biden loses when he actually wins.

Classification Based on Polls

- Accuracy of our classification:
 - `sign()` returns 1 for a positive number, -1 for a negative number, and 0 for 0.

```
1 > poll_pred |>
2   summarize(prop_correct = mean(sign(margin_pred) == sign(margin)))
3
4 # A tibble: 1 × 1
5   prop_correct
6     <dbl>
7 1       0.922
```

- Q: Which states were misclassified by the poll?:

```
1 > poll_pred |>
2   filter(sign(margin_pred) != sign(margin))
3
4 # A tibble: 4 × 8
5   state margin_pred   ev Biden trump   other margin errors
6   <chr>      <dbl> <int> <dbl> <dbl> <dbl> <dbl> <dbl>
7 1 FL          0.0800    29  47.9  51.2  0.0835 -3.36 -3.44
8 2 GA         -1.15     16  49.5  49.2  0.0759  0.236  1.39
9 3 NC          3.95     15  48.6  49.9  0.296  -1.35 -5.30
10 4 NV         -0.350     6  50.1  47.7  0.759   2.39  2.74
```

4/ Time-Series Plot

National Polls

- We often want to show a time series of the national-level polls to get a sense of the popular vote.
 - Useful to bolster the credibility of our prediction!

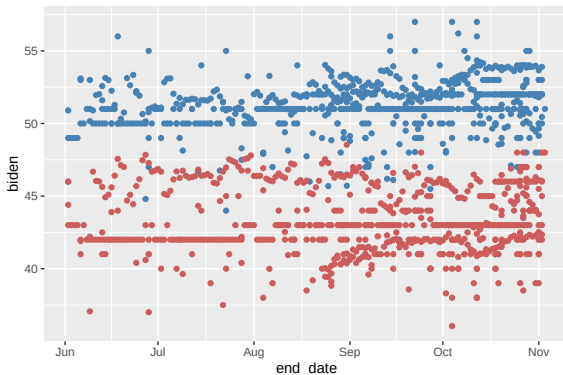
```
1 > national_polls20 <- read_csv("https://bit.ly/3VOUjsa"); national_polls20
2
3 # A tibble: 654 × 5
4   end_date   pollster sample_size biden trump
5   <date>    <chr>      <dbl> <dbl> <dbl>
6 1 2020-11-03 Lake Research      2400    51    48
7 2 2020-11-02 Research Co.      1025    50    42
8 3 2020-11-02 YouGov           1363    53    43
9 4 2020-11-02 Ipsos              914    52    45
10 5 2020-11-02 SurveyMonkey     28240    52    46
11 6 2020-11-02 HarrisX          2297    52    48
12 7 2020-11-02 TIPP             1212   50.4   46.0
13 8 2020-11-02 USC Dornsife      5423   53.9   42.4
14 9 2020-11-01 John Zogby Strategies/EMI Research Solutio. 1008   49.6   43.8
15 10 2020-11-01 Swayable         5174   51.8   46.1
16 # 644 more rows
17 # Use `print(n = ...)` to see more rows
```


Plotting the Raw Results

- Plot the polling results using ggplot2:

```
1 > national_polls20 |>  
2   na.omit() |>  
3   ggplot(aes(x = end_date)) +  
4     geom_point(aes(y = biden), color = "steelblue") +  
5     geom_point(aes(y = trump), color = "indianred")
```

- Fairly messy:



Clean the Mess with Moving Averages

- **Goal:** plot the average of polls in the last 7 days (challenging to do just with `dplyr`).
- Loop over each day in the data and do:
 1. Subset to all polls in the previous 7 days of that day.
 2. Calculate the average of these polls for Biden and Trump.
 3. Save the result as a 1-row tibble.

Dates in R

- You can get R to properly understand dates and do arithmetic with them by storing them as date:

```
1 > head(national_polls20$end_date)
2 [1] "2020-11-03" "2020-11-02" "2020-11-02" "2020-11-02" "2020-11-02" "2020-11-02"
3
4 > head(national_polls20$end_date + 3)
5 [1] "2020-11-06" "2020-11-05" "2020-11-05" "2020-11-05" "2020-11-05" "2020-11-05"
```

- We can convert a string to a date using the **lubridate** package:

```
1 > # Don't forget to install and load the package!
2
3 > "2020-11-03" + 3 ## R doesn't know this is a date yet!
4 Error in "2020-11-03" + 3: non-numeric argument to binary operator
5
6 > lubridate::ymd("2020-11-03") + 3
7 [1] "2020-11-06"
8
9 > lubridate::mdy("11/03/2020") + 3
10 [1] "2020-11-06"
```

Creating a Vector of Dates

- Setup the vector of dates to cover the whole 'window':

```
1 > election_day <- lubridate::ymd("2020-11-03")
2 > all_dates <- seq(from = min(national_polls20$end_date) + 1,
3                     to = election_day,
4                     by = "days")
5 > head(all_dates)
6 [1] "2020-06-03" "2020-06-04" "2020-06-05" "2020-06-06"
7 [5] "2020-06-07" "2020-06-08"
```

Moving Window Loop

- Calculate the moving averages of polls with loop:

```
1 > output <- vector("list", length = length(all_dates)) # empty list to assign the output
2
3 > for (i in seq_along(all_dates)) {
4   this_date <- all_dates[[i]]
5
6   this_week <- national_polls20 |>
7     filter(
8       this_date - end_date >= 0, # this_date is after end_date
9       this_date - end_date < 7  # within a week
10    )
11
12   output[[i]] <- this_week |>
13     summarize(
14       date = this_date,
15       biden = mean(biden, na.rm = TRUE),
16       trump = mean(trump, na.rm = TRUE)
17     )
18 }
```

- Combine all the weekly summaries into a single tibble:

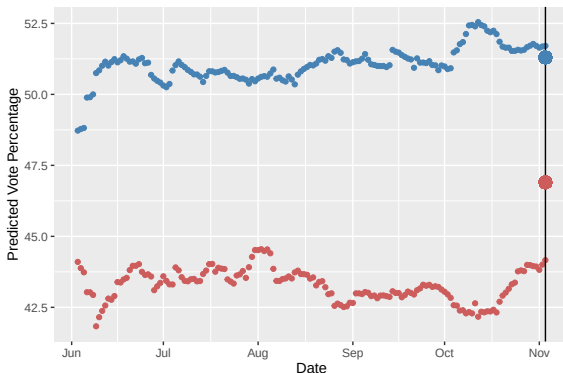
```
1 > output <- bind_rows(output)
```

Examine the Result

```
1 > output
2
3 # A tibble: 154 × 3
4   date      biden trump
5   <date>    <dbl> <dbl>
6 1 2020-06-03  48.7  44.1
7 2 2020-06-04  48.8  43.9
8 3 2020-06-05  48.8  43.7
9 4 2020-06-06  49.9  43.0
10 5 2020-06-07  49.9  43.0
11 6 2020-06-08  50    42.9
12 7 2020-06-09  50.8  41.8
13 8 2020-06-10  50.8  42.2
14 9 2020-06-11  51.0  42.4
15 10 2020-06-12  51.2  42.6
16 # 144 more rows
17 # Use `print(n = ...)` to see more rows
```

Plotting the Moving Averages

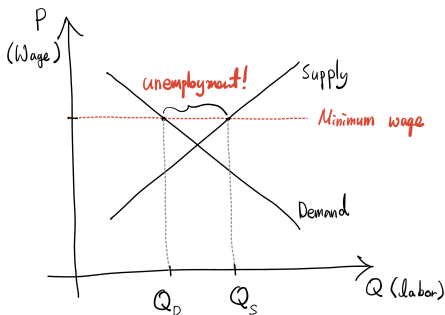
```
1 > output |>  
2   ggplot(aes(x = date)) +  
3     geom_point(aes(y = biden), color = "steelblue") +  
4     geom_point(aes(y = trump), color = "indianred") +  
5     geom_vline(xintercept = election_day) +  
6     geom_point(aes(x = election_day, y = 51.3), color = "steelblue", size = 4) +  
7     geom_point(aes(x = election_day, y = 46.9), color = "indianred", size = 4) +  
8     labs(x = "Date", y = "Predicted Vote Percentage")
```



5/ Review: Causal Effect Estimation

Minimum Wage Study

- **Q: Does increasing the minimum wage affect employment?**
 - Economists worry that requiring higher wages will lead employers shifting workers from full time to part time.
 - But that's a theoretical concern, can we give empirical evidence?
- Canonical study: Card and Krueger (1994) on minimum wage laws in NJ
 - In 1992, NJ raised minimum wage from \$4.25/hr to \$5.05/hr.
 - What is the effect of this change?



What Can We Do?

- **Does increasing the minimum wage affect employment?**
- Three ways to analyze data (i.e., designs):
 - Cross-sectional: Compare NJ employment to neighbor PA employment in 1993 (after) $\rightsquigarrow$ limited by **unmeasured confounding**.
 - Before-and-after: Compare changes in NJ employment between 1991 (before) and 1993 (after) $\rightsquigarrow$ limited by **time-varying confounding**.
 - **Difference-in-differences**: Compare changes in NJ employment between 1991 (before) and 1993 (after) to changes in PA in the same period.
- **Q**: RCT or observational study?

Card & Krueger Data

Name	Description
chain	Name of the fast-food restaurant chain
location	Location of the restaurant
wageBefore	Average wage at the restaurant before NJ minimum wage law
wageAfter	Average wage at the restaurant after NJ minimum wage law
fullBefore	Number of full-time employees before NJ minimum wage law
fullAfter	Number of full-time employees after NJ minimum wage law
partBefore	Number of part-time employees before NJ minimum wage law
partAfter	Number of part-time employees after NJ minimum wage law

- Import data:

```
1 > minwage <- as_tibble(read.csv(url("https://bit.ly/3QZmodH"))); head(minwage)
2
3 # A tibble: 6 × 8
4   chain    location wageBefore wageAfter fullBefore fullAfter partBefore partAfter
5   <chr>    <chr>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
6 1 wendys  PA           5      5.25     20       0       20      36
7 2 wendys  PA          5.5     4.75      6      28      26      3
8 3 burgerking PA          5      4.75     50      15      35     18
9 4 burgerking PA          5       5      10      26      17      9
10 5 kfc    PA          5.25     5       2       3       8     12
11 6 kfc    PA          5       5       2       2      10      9
```

Creating a Treatment Vector

```
1 > minwage <- minwage |>
2   mutate(
3     state = if_else(location == "PA", "PA", "NJ") ## PA is control
4   ); head(minwage, 2)
5
6 # A tibble: 2 × 11
7   chain location wageBefore wageAfter fullBefore fullAfter partBefore partAfter state
8   <chr> <chr>      <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl> <chr>
9 1 wendys PA          5      5.25      20      0       20      36 PA
10 2 wendys PA        5.5      4.75       6     28      26      3 PA
11 # 2 more variables: full_prop_after <dbl>, full_prop_before <dbl>
```

- Prob the number of restaurants by location:

```
1 > minwage |>
2   count(location)
3
4 # A tibble: 5 × 2
5   location      n
6   <chr>    <int>
7 1 PA        67
8 2 centralNJ  45
9 3 northNJ   146
10 4 shoreNJ   33
11 5 southNJ   67
```

Estimating the Difference-in-Differences

```
1 > minwage |>
2   group_by(state) |>
3   mutate(full_prop_after = fullAfter / (fullAfter + partAfter),
4           full_prop_before = fullBefore / (fullBefore + partBefore)) |>
5   summarize(trend = mean(full_prop_after) - mean(full_prop_before)) |>
6   pivot_wider(
7     names_from = state,
8     values_from = trend
9   ) |>
10  mutate(DID = NJ - PA)
11
12 # A tibble: 1 × 3
13     NJ      PA      DID
14   <dbl> <dbl> <dbl>
15 1 0.0239 -0.0377 0.0616
```

- **Interpretation:** minimum wage laws increased the proportion of full-time employees in NJ by 6.16% points.
 - Assumption: if trends in PA are a good proxy for trends in NJ if it didn't enact a minimum wage law.

Enjoy Your Weekends! :)

Contact Information:

jaewon.yoo@iss.nthu.edu.tw

<https://j1yoo.github.io/>

